1. Cum un graf conex are cel putin un arbore de acoperire? Ca daca am lua un subgraf (adica sa scoatem cel putin o muchie), nu ar mai fi interconectate toate nodurile, deci nu ar mai fi arbore de acoperire.
–ce zici tu se intampla daca grafu era deja arbore; da oricum, e **graf partial** si daca nu scoti muchii (subgraf e daca tai noduri)
-- da, un arbore de acoperire are oricum n - 1 muchii (orice arbore cu n noduri are n-1 muchii) si atunci daca ai porni cu un graf cu n-1 muchii, nu exista un arbore de acoperire daca muchiile nu conecteaza toate nodurile intre ele (se poate forma o padure de arbori de acoperire, cate unul pentru fiecare componenta conexa). Gandeste-te la graful neorientat cu muchiile [1, 2], [2, 3], [1, 3], pot oricand sa scap de una dintre muchii ca nodurile raman oricum conectate

2. Avem si A*, Minimax si Minheap/Maxheap?
A*, minimax am vazut ca s-a mai dat la subiectul 1 mai mult/max 2 cu intrebari de intelegere. Minheap/maxheap nu prea am vazut. La A* si minimax recomand sa va uitati ca sunt simplu de calculat (la minimax se face doar minimul /maximul dintre valorile copiilor in functie de nivel).

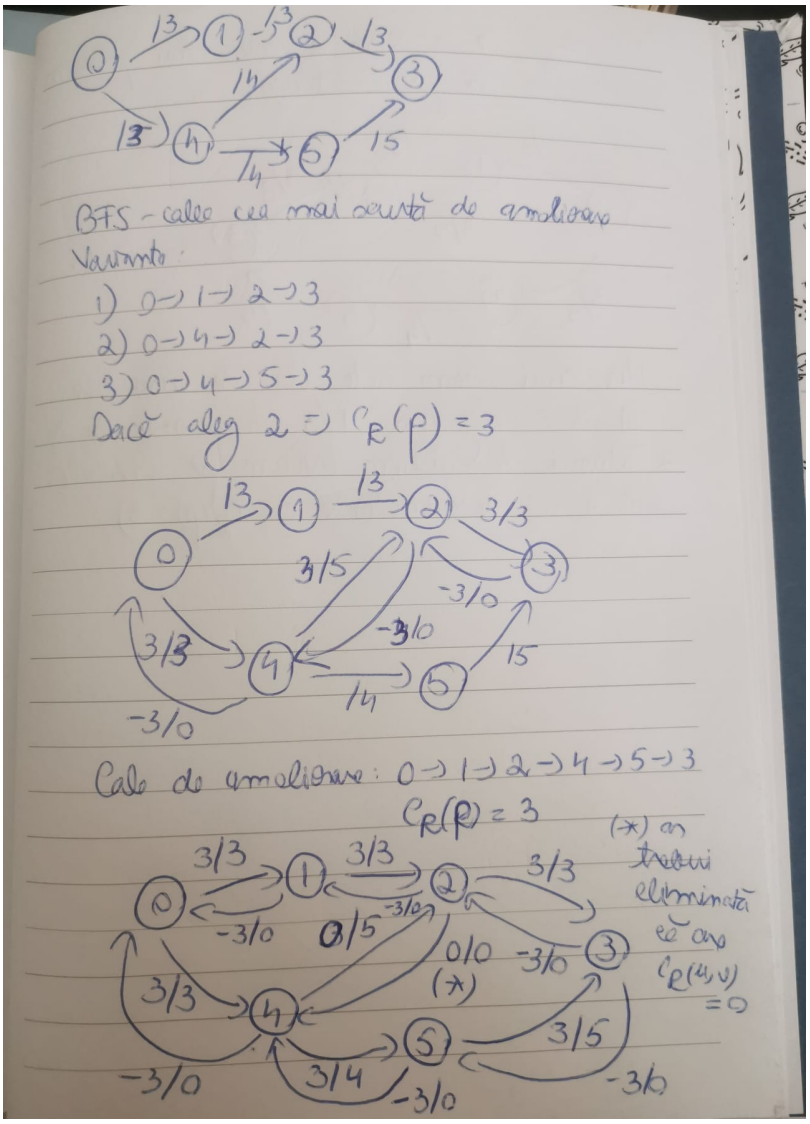
Asta scrie pe ocw: “În anul universitar 2021-2022, acest material este studiu de caz individual și opțional.” Ah, ok, uitasem atunci nu cred ca ar trebui sa intre. Am intrebat echipa si revin cu raspuns
Update: intra, pe ocw se refera strict la ce se face la lab :(daca le-ati facut la curs si nu v-a zis ca nu intra atunci se poate sa apara

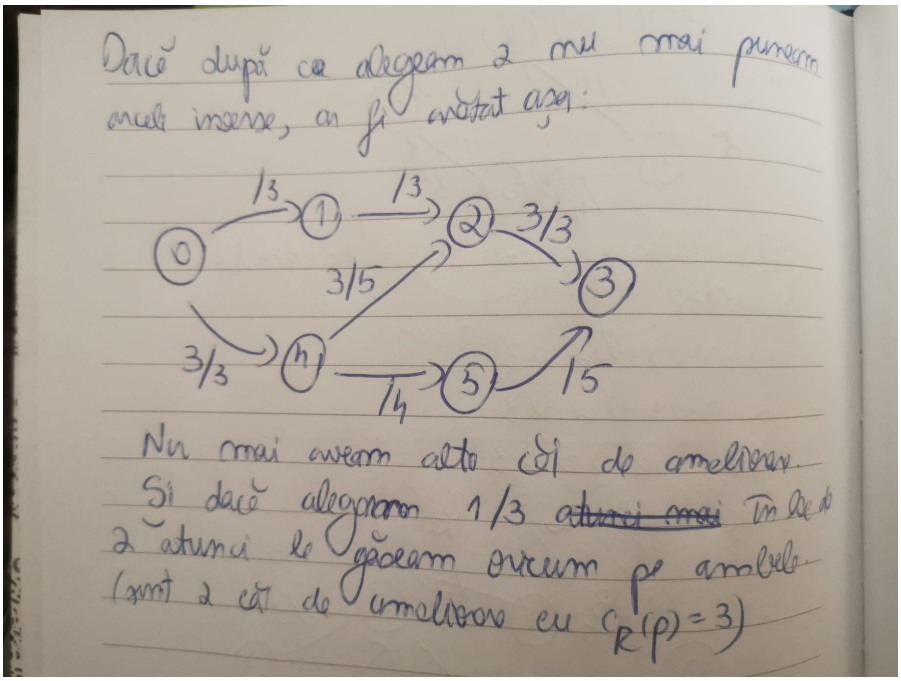
3. Daca trebuie sa gasim cateva din drumurile de ameliorare la flux maxim, putem sa luam orice varianta de drum pe care o face BFS-ul? Da, doar sa nu uitati sa actualizati fluxul si pentru muchiile care nu exista in graful initial ca sa nu obtineti solutii gresite.
Cum adica muchiile care nu erau in graful initial? Imediat sa ma gandesc la un exemplu =))
~ teorie
La flux Ford Fulkerson (si Edmonds Karp oricum) se face parcurgerea pe reteaua reziduala care contine arcuri reziduale, doar ca le pastrezi numai pe cele cu CR(u, v) > 0 (capacitatea reziduala)
Capacitatea reziduala CR(u, v) e diferenta dintre capacitate C(u, v) si flux f(u, v). Daca muchia nu e in graful initial inseamna ca C(u, v) = 0 (nu pot sa duc nimic pe muchia aia) deci CR(u, v) = -f(u,v) = f(v, u) (la flux in general f(u, v) = -f(v, u): daca duc 2 unitati de la u la v e ca si cum as duce -2 de la v la u) si atunci CR(u, v) poate fi pozitiv daca f(v, u) este pozitiv.
Si de asta duce afli o cale:

- Afli capacitatea reziduala a caii
- Parcurgi muchiile care exista in cale si aduni CR(p) (capacitatea reziduala a caii) la f(u, v) si o scazi la f(v, u)

~exemplu

Daca ai timp, ai putea sa pui o poza, te rog, cu un exemplu concret cum s-ar face? Yess, imediat ca sunt de pe telefon, revin =)) Multumesc! Cu draag <3





4. Daca vrem sa gasim arborele minim de acoperire, daca facem cu Kruskal sau cu Prim, obtinem la fel?
In general, cred ca pot sa difere mai ales ca la prim depinde si de sursa de la care pornesti

5. Numarul maxim de puncte de articulatie intr-un graf cu n noduri si m arce?
Nu știu dacă se vrea și mai detaliat de atât, dar eu as lua pe cazuri după m.

C1) m = 0
Avem n componente conexe pentru ca fiecare nod formează una
|1| |2|... |n|
Sunt 0 puncte de articulație pentru ca dacă scot un nod micsorez numărul de componente conexe (rămân n-1)

C2) m = n - 1 și graf conex
|1|--|2|--...--|n|
Aici sunt n-2 puncte de articulație pentru ca orice nod din "interior" sparge graful în 2 componente conexe:
|1|--|2|--|i - 1| |i + 1|--...--|n| (dacă as scoate i cu 1 < i < n)
Dacă încerc sa scot 1 tot o componenta conexa o sa am:
|2|--...--|n|
La fel dacă l-aș scoate pe n

Și ar mai fi cazurile mai mici când am numai 1 / 2 noduri
Pentru 1 nod: 0 puncte de articulație
Pentru 2 noduri: tot 0 (orice nod as scoate, tot o singura componenta conexa cu un singur nod o sa rămâna)

Deci, cred ca minim 0 pot sa fie, maxim n-2 dacă n>2.

6. Determinați numărul minim de înmulțiri de scalari necesare pentru înmulțirea următoarelor matrici, precum și parantezarea necesară pentru a ajunge la acest număr optim: A1(4 linii, 10 coloane), A2(10, 3), A3(3, 12), A4(12, 20), A5(20, 7).

–teorie ish
Dacă ai 2 matrici A de dimensiune (m, n) și B (n, p) atunci ai m*n*p înmulțiri scalare și rezultatul are dimensiunea (m, p).
Dacă ai avea 3 matrici

A	*	B	*	C
(1, 2)		(2, 4)		(4,8)

atunci ai 2 variante de parantezari:

- (AB) C: 1*2*4=8 pentru AB și 1*4*8 = 32 pentru (AB)C (AB o sa fie de dimensiune (1,4)) deci 40 in total
- A(BC): 2*4*8=64 pentru BC și 1*2*8 = 16 pentru A(BC) (BC o sa fie de dimensiune (2,8)) deci 80 in total

Aici sunt 5 deci sunt mai multe combinații și poți sa folosești o dinamica.
dp[i][j] = numărul minim de înmulțiri scalare pentru Ai *... *Aj

Cazul de baza:

- O singura matrice: 0 înmulțiri ca nu ai cu ce sa o înmulțești (dp[i][i] = 0)
- 2 matrici Ai(m, n), A(i+1)(n, p): dp[i][i+1] = m*n*p

Cazul general: încerci sa spargi secvența în 2
Ai *... *Aj = (Ai *.. * Ak) * (A(k+1) *.. * Aj) si iei minimul deci dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j] + m * n * p) cu Ai *.. *Ak de dimensiune (m, n) și A(k+1) *... * Aj de dimensiune (n, p)

–exercitiu
Acum pe ex asta începi de la cele mai mici secvente și le tot maresti:

// A1(4, 10), A2(10, 3), A3(3, 12), A4(12, 20), A5(20, 7).

dp[1][1] = dp[2][2] = dp[3][3] = dp[4][4] = dp[5][5] = 0

// A1*A2(4, 3), A2*A3(10, 12), A3*A4(3, 20), A4*A5(12, 7)

dp[1][2] = 4 * 10 * 3 = 120
dp[2][3] = 10 * 3 * 12 = 360
dp[3][4] = 3 * 12 * 20 = 720
dp[4][5] = 12 * 20 * 7 = 1680

// A1*A2*A3(4,12), A2*A3*A4(10, 20), A3*A4*A5(3,7)

dp[1][3] = min(dp[1][1] + dp[2][3] + 4*10*12, dp[1][2] + dp[3][3]+4*3*12) = min(360 + 480,120 + 144) = min(840, 264) = 264
dp[2][4] = min(dp[2][2] + dp[3][4] + 10 * 3 * 20, dp[2][3] + dp[4][4] + 10 * 12 * 10) = min(720 + 600, 360 + 1200) = min(1320, 1560) = 1320
dp[3][5] = min(dp[3][3] + dp[4][5] + 3 * 12 * 7, dp[3][4] + dp[5][5] + 3 * 20 * 7) = min(1680 + 252, 720 + 420) = min(1932, 1140) = 1140

// A1*A2*A3*A4(4, 20), A2*A3*A4*A5(10, 7)

dp[1][4] = min(dp[1][1] + dp[2][4] + 4 * 10 *20, dp[1][2] + dp[3][4]+ 4 * 3 * 20, dp[1][3] + dp[4][4] + 4 * 12 * 20) = min(1320 + 800, 120 + 720 + 240, 264 + 960) = min(2120, 1080, 1224) = 1080
dp[2][5] = min(dp[2][2] + dp[3][5] + 10 * 3 * 7, dp[2][3] + dp[4][5]+ 10 * 12 * 7, dp[2][4] + dp[5][5] + 10 * 20 * 7) = min(1140 + 210, 360 + 1680 + 840, 1320 + 1400) = min(1350, 2880, 2720) = 1350

dp[1][5] = min(dp[1][1] + dp[2][5] + 4 * 10 * 7, dp[1][2] + dp[3][5]+ 4 * 3 * 7, dp[1][3] + dp[4][5] + 4 * 12 * 7, dp[1][4] + dp[5][5] + 4 * 20 * 7) = min(1350 + 280, 120 + 1140 + 84, 264 + 1680 + 336, 1080 + 560) = min(1630, 1344, 2280, 1640) = 1344

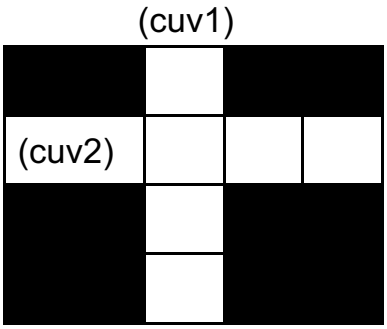
Numarul minim de înmulțiri scalare = 1344
Parantezarea: (A1*A2) * ((A3*A4)* A5)
(pentru ce varianta am obținut minimul)

dp[1][2] + dp[3][5] +...
Iar dp[3][5] e de la dp[3][4] + dp[5][5] +..

Cred ca asta ar fi, sper sa nu fi bușit vreun calcul :(

7. Să se spună ce schemă de algoritm făcută la curs poate să determine, cât mai eficient posibil, dacă un careu de rebus are o asignare posibilă de cuvinte (nu se consideră definițiile cuvintelor). Să se schițeze ideea rezolvării. Se presupune că există un dicționar în care cuvintele sunt reprezentate prin vectori de caractere. (un careu de rebus este o matrice **m x n** cu unele celule goale. Toate celelalte celule trebuie să primească câte o literă astfel încât secvențele maximale de celule pe orizontală și verticală să fie cuvinte valide).

Cred ca o varianta ar fi cu backtracking si exprimarea unor restrictii.
Daca ar fi tabla:



Atunci trebuie gasite 2 cuvinte care au o suprapunere (litera de pe pozitia 2 e comuna in ambele).
Pentru fiecare cuvant ar fi cate o restrictie unara legat de dimensiune, de unde incepe si eventual pozitie:

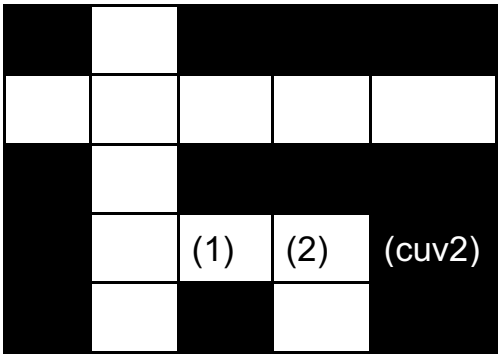
- Pentru cuv1 secventa care incepe din (0, 1) de lungime 4 in jos trebuie sa fie un cuvant valid (se poate verifica usor daca un cuvant este valid ⇔ daca apartine sau nu dictionarului)
- Pentru cuv2 asemanator: secventa care incepe de la (1, 0) de lungime tot 4 care se continua la stanga trebuie sa fie un cuvant valid

Si mai trebuie si niste restrictii binare acolo unde se suprapun:

- Al doilea caracter din cuv1 trebuie sa fie egal cu al doilea din cel din cuv2

Astea ar fi restrictiile. Si ajuta AC3 pentru ca daca as avea 3 cuvinte de genul:

(cuv1)



(cuv3)

Daca as schimba (1) atunci nu influenteaza direct cuv1(nu are sens sa il reevaluez). Dar daca schimb (3) atunci sunt 2 cuvinte care depind de el (cuv2, cuv3) si atunci se reverifica numai nodurile care depind de acea pozitie.

8. Comparați algoritmul discutat la curs pentru construirea unui arbore de codificare Huffman cu alternativa de a-l construi cu algoritmul de la construirea arborilor optimi la căutare. -> Intrebare: Daca construim arborele Huffman cu AOC, cum putem afla atunci codificarea caracterelor?

-- legat de codificare poti sa o pastrezi la fel

Ce e caracteristic la Huffman nu e neaparat cum ii codifici (0 la stanga/1 la dreapta) ci mai mult ordinea si cum formezi nodurile. Poti sa pastrezi codificarea si la arbori optimi de cautare doar sa ai grija la decodificare. Daca ai arborele:

|a|

|b| |d|

|c|

Lui a pot sa ii asignez un caracter special (root), b ar fi 0, d 1 si c 01. Daca primesc sirul 01 pentru decodificare atunci am 2 variante: fie bd(b e 0, d e 1) fie c(c e 01). Ca sa stiu care e atunci pot sa ma folosesc de un separator: 0|1 sau sa pun restrictii (fac match pe cea mai scurta/lunga varianta dar asta nu garanteaza ca e corect mereu).

La Huffman:

- Faci cate un arbore per cheie
- Alegi 2 arbori cu frecventa minima & ii unesti & adaugi noul arbore la multimea de arbori
- Faci asta pana ce mai ramai cu un singur arbore

Dar atunci cand ii construiesti asa cheile o sa fie numai in frunze si asta cred ca e o diferenta mare fata de arbori optimi de cautare. La arbori optimi de cautare o sa ai chei si in noduri interne.

Pentru sirul abbccc daca faci Huffman obtii ceva de genul:

```
       |:6|
 |c:3|      |:3|
         |a:1| |b:2|
```

Iar cu arbori optimi de cautare o varianta ar fi poate asta:

```
    |c|
  |b|
|a|
```

Pentru ca sunt arbori de cautare atunci poti cauta si toate nodurile pentru care valoarea e mai mica/mare decat ceva, nu numai egalitate. Tot aici numarul de noduri e mai mic decat la huffman (la huffman se creeaza multe noduri intermediare) si se minimeaza timpul mediu de cautare pentru o secventa de caractere.

9. La S I, ex. 1, 2020, dfs-ul putea sa fie si 6, 5, 3, 4 etc?
Da, e super ok si cu 5 primul doar sa nu se ceara in enunt vreo ordine anume

10. Să se spună ce algoritm făcut la curs poate să determine eficient care este drumul de lungime maximă într-un graf aciclic orientat cu lungimi pe arce și să se schițeze ideea folosirii acestuia și modificările necesare pentru rezolvare.

Daca e DAG (aciclic & orientat) atunci de obicei te cam ajuta o sortare toplogica. Ca sa existe o cale de la u la v ar trebui ca u sa apara inaintea lui v in sortarea topologica deci poti sa te folosesti de ordinea data de topsort pentru a determina mai eficient drumurile.
Poti sa faci un topsort ca sa obtii ordinea si dupa parcurgi nodurile in acea ordine (nu mai ai nevoie de coada de la Dijkstra) si relaxezi muchiile unde e cazul. Daca e de cost minim e aceeasi conditie ca la Dijkstra (d[node] + w[node][neigh] < d[neigh]). Daca vrei de cost maxim (aici lungime maxima e acelasi lucru pentru

ca pe arce sunt lungimi) atunci trebuie inversata conditia si relaxezi muchia cand gasesti o lungime mai mare (d[node] + w[node][neigh] > d[neigh])

Un mini pseudocod:

```
// initializare d cu -inf/inf in functie de ce se cauta
sorted = topsort(G)
For node in sorted
    For neigh in adj[node]
        If cond(...) // conditia de relaxare cu </> in functie de ce se cere
          d[neigh] = d[node] + w[node][neigh]
          p[neigh] = node
```